

Reviewer Pack — Minimal p-Adic Valuation Rank and Frege Proof Depth Inequality...

Entry 4da98be81a60 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 08:20:17 UTC

Minimal p-Adic Valuation Rank and Frege Proof Depth Inequality

Entry ID: 4da98be81a60 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 08:20:17 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis (Valuation Theory) **Field B** (complexity object): Frege Proof Complexity

Statement:

For every CNF φ with n variables, the minimal p-adic valuation rank (r_φ) of its associated polynomial is linearly correlated with its Frege proof depth $d(\varphi)$, such that $r_\varphi = \Omega(d(\varphi))$. Additionally, for all instances with property P, property Q holds, where P is defined as 'the polynomial has a valuation ring extension of characteristic p^k ' and Q is defined as 'the Frege proof depth is at least $k \log_p(n) + 1$ '.

Rationale (proposer's reasoning):

Valuation theory provides a framework for understanding the arithmetic properties of rings, which could potentially reveal non-trivial relationships with proof complexity. The correlation between valuation rank and Frege proof depth may expose structural insights into the complexity of proving satisfiability.

Taxonomy category: Resolution Proof Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 85afcf779dec0d70

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a CNF φ with n variables, the minimal p-adic valuation rank r_φ and Frege proof depth $d(\varphi)$ satisfy: if $r_\varphi \geq \Omega(d(\varphi))$, AND for all instances with P (valuation ring extension of characteristic p^k), Q (Frege proof depth at least $k \log_p(n) + 1$) holds. Support is indicated by a correlation coefficient ≥ 0.7 over 30 seeds, and falsification requires an instance where P is false and Q is true.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(random.randint(1, n)):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            clauses.append(clause)
        return clauses

    def p_adic_valuation_rank(poly):
        # Placeholder implementation
        return random.randint(1, n)

    def frege_proof_depth(cnf):
        # Placeholder implementation
        return random.randint(10, 50)

    correlation_coefficient = 0.0
    instances_tested = 0
    n_max = 0
    conjecture_holds = True
    counterexample = ""

    for _ in range(30):
        n = random.choice([5, 10, 15, 20, 30, 40])
        cnf = generate_cnf(n)
        r_phi = p_adic_valuation_rank(cnf)
        d_phi = frege_proof_depth(cnf)
```

```

    if n > n_max:
        n_max = n

    correlation_coefficient += (r_phi - 1) * (d_phi - 25) / 20
    instances_tested += 1

    p_k = random.randint(1, int(math.log(n, 2)))
    k_log_n_plus_1 = math.ceil(k_log_n + 1)

    if r_phi < d_phi and not conjecture_holds:
        counterexample = f"r_phi={r_phi} < d_phi={d_phi}"
        break
    elif r_phi >= d_phi and p_k > k_log_n_plus_1 and conjecture_holds:
        counterexample = f"p_k={p_k} > {k_log_n_plus_1}, but r_phi={r_phi} >= d_phi={d_phi}"
        conjecture_holds = False

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient / instances_tested,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif first_failing_seed is not None:
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6621c228.py", line 78, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6621c228.py", line 55, in run_trial
    k_log_n_plus_1 = math.ceil(k_log_n + 1)
                    ^^^^^^^

```

NameError: name 'k_log_n' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture’s validity. | next: Review and debug the test code to ensure it can run successfully and produce results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14410
2	preregistration	ollama_remote	glm4:latest	0	0	9798
3	novelty	ollama_remote	glm4:latest	0	0	8944
4	novelty	ollama_remote	glm4:latest	0	0	9024
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14656
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15696
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11110
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10094
9	judge	ollama_remote	glm4:latest	0	0	25027

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 118758 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/4da98be81a60.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4da98be81a60.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4da98be81a60.tar.gz (if generated)